Real Estate Investment Advisor: Project Documentation

Author: *Saish Bhalchandra Bore*

Data preprocessing, exploratory analysis, machine learning, experiment tracking and a Streamlit application

1. Overview

The Real Estate Investment Advisor helps a user judge a property before buying it. Given the details of a property, it answers two questions:

1. **Is this a good investment?** (classification)
2. **What will the property be worth in 5 years?** (regression)

The models sit behind a Streamlit web application, and MLflow records every training run so the models can be compared. The project uses a dataset of 250,000 Indian property listings.

One point runs through this whole document: **the dataset appears to be synthetic, and both prediction targets were defined by rules written for this project.** Sections 4, 6 and 9 explain what that means for the results.

2. Dataset

The file `india_housing_prices.csv` has 250,000 rows and 23 columns:

- **Location:** State, City, Locality
- **Property:** Property_Type, BHK, Size_in_SqFt, Year_Built, Age_of_Property, Furnished_Status, Floor_No, Total_Floors, Facing
- **Price:** Price_in_Lakhs, Price_per_SqFt
- **Neighborhood:** Nearby_Schools, Nearby_Hospitals, Public_Transport_Accessibility
- **Features and listing:** Parking_Space, Security, Amenities, Owner_Type, Availability_Status, ID

Initial checks found no missing values and no duplicate rows or duplicate IDs.

3. Data preprocessing

All preprocessing is in `src/step1.py`.

3.1 Problems found and how they were handled

Problem	Finding	Action
Price_per_SqFt	Stored in lakhs per sqft and rounded to 2 decimals; 2,534 rows showed 0.00	Recomputed in rupees: Price_per_SqFt_INR = Price x 100,000 / Size
Floors	116,304 rows (46.5%) had Floor_No above Total_Floors	Set Total_Floors to at least Floor_No and added a Floor_Data_Fixed flag
Amenities	325 text variants that were really 5 amenities in different orders	Split into five 0/1 columns plus Amenity_Count
Yes/No columns	Parking_Space and Security stored as text	Converted to 1/0
Redundant columns	ID is only a label; State is fully determined by City; Locality has 500 names reused in every city; Age_of_Property plus Year_Built is always 2025	Dropped

3.2 Encoding

- **Ordered columns** became numbers: Public_Transport_Accessibility (Low, Medium, High as 1, 2, 3) and Furnished_Status (Unfurnished, Semi-furnished, Furnished as 0, 1, 2).
- **Unordered columns** were one-hot encoded: Property_Type, Facing, Owner_Type, Availability_Status and City.

3.3 Feature engineering

- **Infrastructure_Score:** the average of schools / 10, hospitals / 10 and transport / 3, giving a value from 0 to 1.
- **Size_per_BHK:** size divided by number of bedrooms.
- **Price_vs_City_Median:** price per sqft divided by the median for the property's city.
- **Floor_Position:** floor divided by total floors. After the floor repair, about half of all rows sit at exactly 1.0, so this feature is unreliable and was left out of the models.

3.4 The two targets

Neither target exists in the raw data, so both were defined:

- **Future_Price_5Y** = current price x (1 + growth rate)^5, where the growth rate is 5% plus up to 5% more depending on Infrastructure_Score (about 5.9% to 10% a year).
- **Good_Investment** = 1 when the property's price per sqft is at or below its city median **and** its Infrastructure_Score is at or above the median. This gives 64,900 positive rows (26%) and 185,100 negative rows.

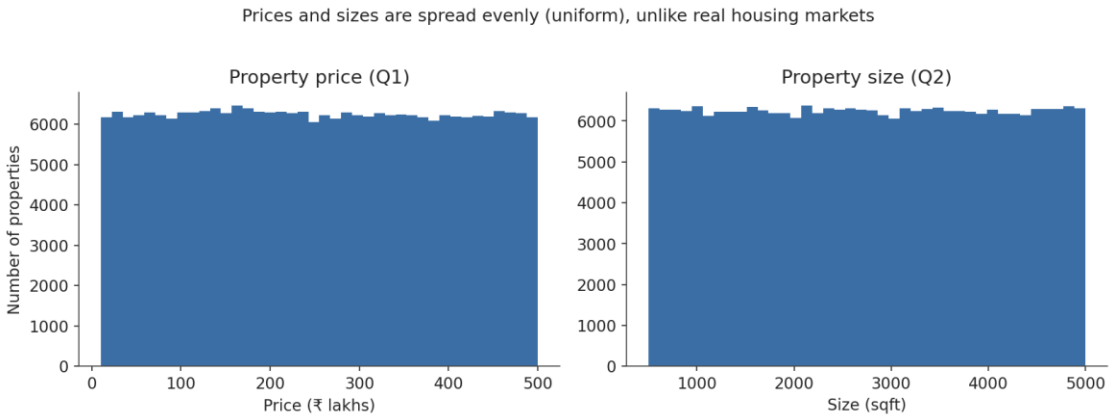
3.5 Split and scaling

The data was split 80/20 into 200,000 training rows and 50,000 test rows, stratified so that 25.96% of both sets are good investments. A StandardScaler was fitted on the training set

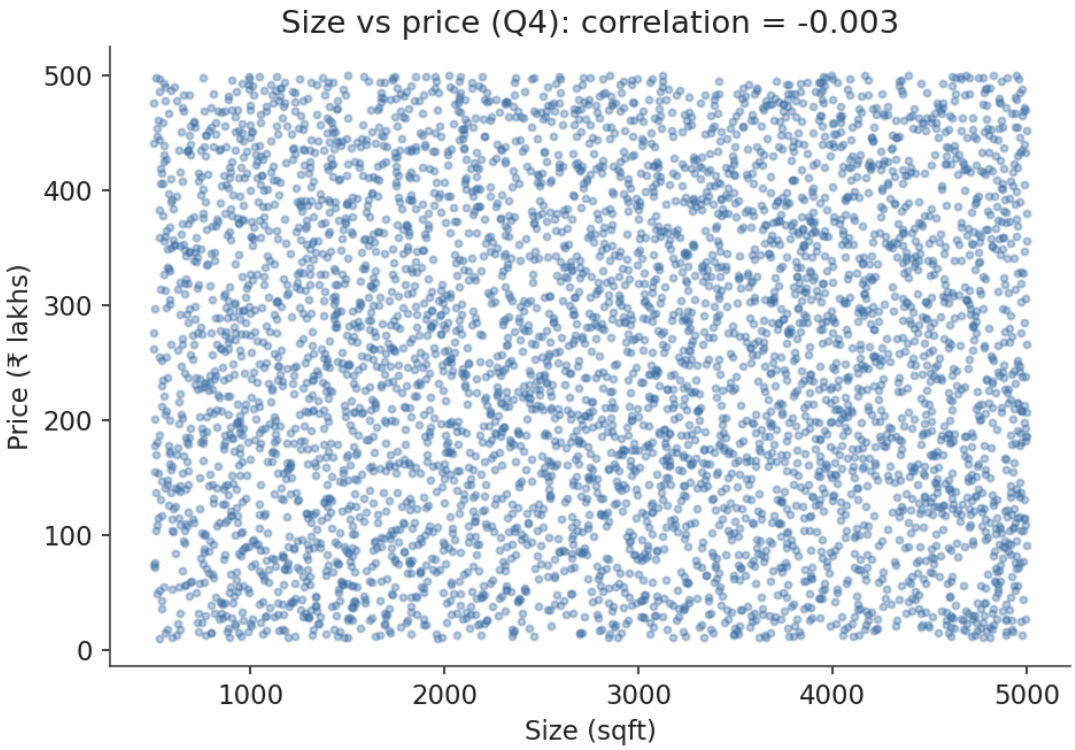
only, applied to the test set, and saved so the app can scale user input the same way. The final table has 76 columns, of which 71 are model inputs.

4. Exploratory data analysis

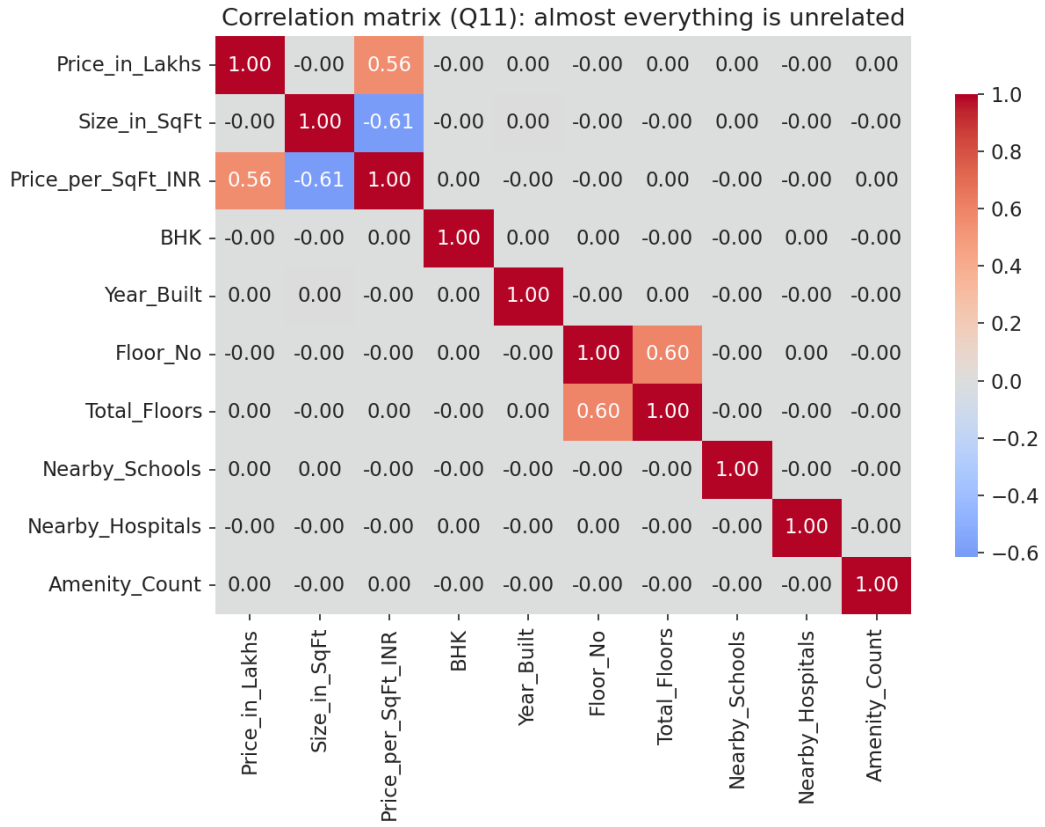
The 20 questions are answered in `notebooks/eda.ipynb`.



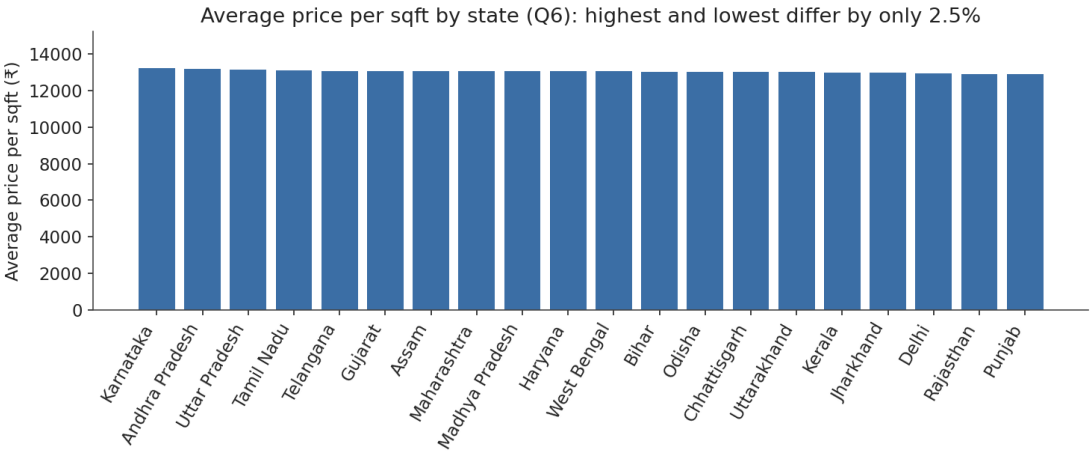
Price and size distributions



Size versus price



Correlation matrix



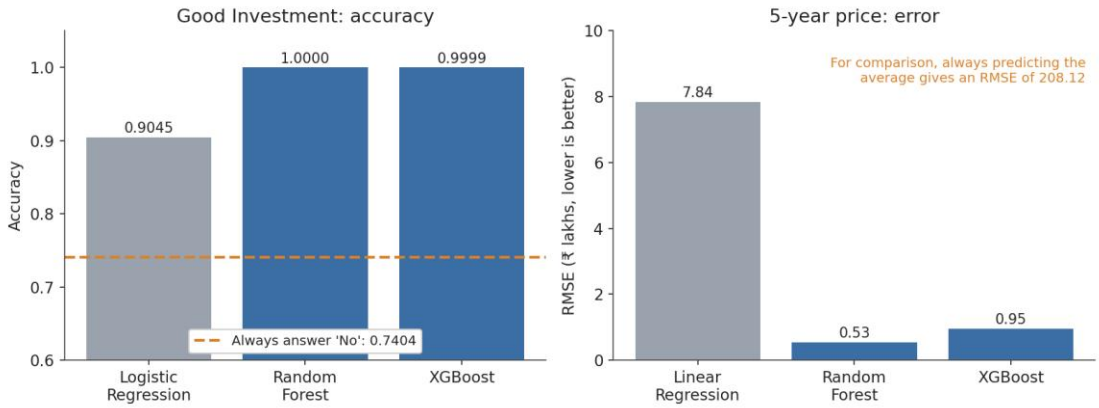
Price per sqft by state

Question	Finding
Q1 Price distribution	Uniform between 10 and 500 lakhs; every price band holds about the same number of properties
Q2 Size distribution	Uniform between 500 and 5,000 sqft
Q3 Price per sqft by property type	Medians of 9,245 (Apartment), 9,305 (Independent House) and 9,190 (Villa) rupees, within about 1.5%
Q4 Size vs price	No relationship; correlation is -0.003
Q5 Outliers	None in price or size; 19,723 (7.9%) in price per sqft, caused by small properties with high prices. Kept and flagged
Q6 Price per sqft by state	All 20 states between 12,931 and 13,252 rupees per sqft, a gap of about 2.5%
Q7 Average price by city	All 42 cities between 250.8 and 258.5 lakhs
Q8 Median property age by locality	Between 16 and 21 years in every locality, most at 18 to 19
Q9 BHK mix by city	Each BHK type is about 20% of listings in every city
Q10 Trend in top 5 localities	No consistent trend; zigzag consistent with sampling noise
Q11 Correlation matrix	All near zero except mechanical relationships: price per sqft with price (+0.56) and size (-0.61), and Floor_No with Total_Floors (+0.60, caused by the floor repair)
Q12-Q13 Schools and hospitals	No effect on price per sqft
Q14 Furnishing	13,034 (Furnished), 13,093 (Semi-furnished), 13,048 (Unfurnished) rupees per sqft
Q15 Facing	East 13,024, North 13,025, South 13,044, West 13,140 rupees per sqft; under 1% apart
Q16 Owner type	Broker 83,479, Owner 83,268, Builder 83,253
Q17 Availability	Under Construction 125,035, Ready to Move 124,965
Q18 Parking	No effect on price or price per sqft
Q19 Amenities	Differences under 1% (for example Gym +98 rupees per sqft, Playground -23); noise
Q20 Public transport	No effect on price per sqft. The share rated a good investment rises from 10.5% (Low) to 27.4% (Medium) to 39.9% (High), but only because transport is part of the rule that defines the label

Overall conclusion: apart from relationships created by how columns are calculated or by this project's own rules, the features in this dataset are independent of one another and of price.

5. Models and results

Both tasks use the same 71 inputs. Three models were compared for each task and evaluated on the 50,000 held-out test rows.



Model comparison

5.1 Good Investment (classification)

Always answering "No" scores 0.7404 accuracy.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.9045	0.8304	0.7941	0.8119	0.9659
Random Forest	1.0000	1.0000	1.0000	1.0000	1.0000
XGBoost	0.9999	0.9998	1.0000	0.9999	1.0000

The Random Forest classified all 50,000 test properties correctly (37,020 "No" and 12,980 "Yes"). Logistic Regression is lower because the label is a rule with thresholds, which tree-based models can reproduce and a straight-line model can only approximate.

5.2 Price after 5 years (regression)

Always predicting the average gives an RMSE of 208.12 lakhs.

Model	RMSE (lakhs)	MAE (lakhs)	R2
Linear Regression	7.8365	5.5155	0.9986
Random Forest	0.5345	0.3795	1.0000
XGBoost	0.9536	0.7893	1.0000

In the best model, Price_in_Lakhs accounts for 99.4% of feature importance and Infrastructure_Score for 0.6%. This matches how the target was built.

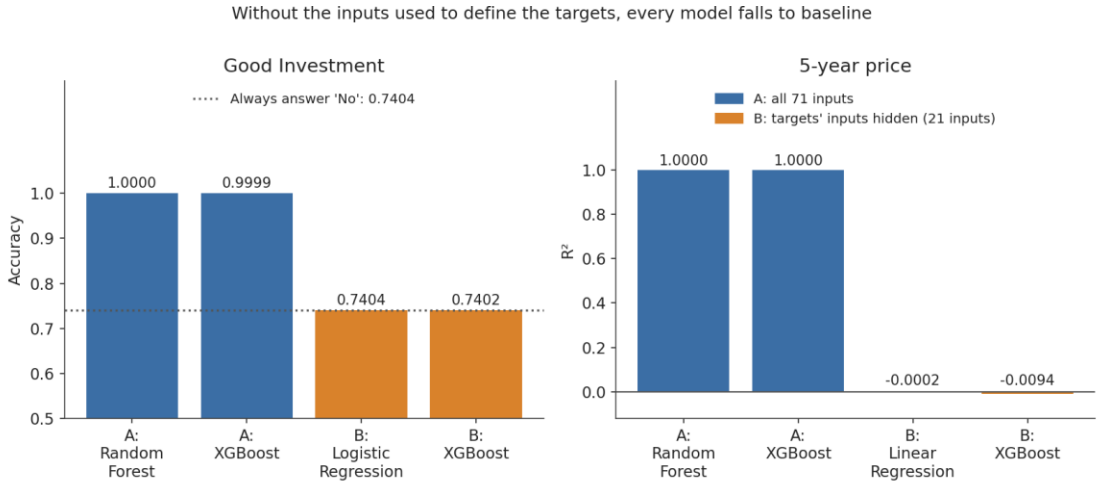
5.3 Experiment tracking

Every model was logged to MLflow under the experiment `Real_Estate_Investment_Advisor`, with its settings, metrics and saved model file (`src/step4_mlflow.py`). The runs are compared in the

MLflow dashboard. The best classifier and regressor (both Random Forest) are saved in `models/` for the app.

6. Option B experiment: removing the targets' inputs

Because the targets were built from a few columns, the high scores in Section 5 could simply mean the models rediscovered those rules. To test this (`src/step6_option_b.py`), every input used to build the targets was hidden: price, size, price per sqft, price versus city median, size per BHK, the infrastructure score, schools, hospitals, transport and city. That left 21 inputs: BHK, year built, furnishing, floors, parking, security, amenities, property type, facing, owner type and availability.



Option A versus Option B

Task	Model	Result	Baseline
Good Investment	Logistic Regression	Accuracy 0.7404, F1 0.0000, ROC-AUC 0.5001	Accuracy 0.7404
Good Investment	XGBoost	Accuracy 0.7402, F1 0.0006, ROC-AUC 0.4991	Accuracy 0.7404
5-year price	Linear Regression	RMSE 208.14, R2 -0.0002	RMSE 208.12
5-year price	XGBoost	RMSE 209.09, R2 -0.0094	RMSE 208.12

Without the targets' own inputs, every model fell to baseline: an ROC-AUC of about 0.5 means no better than guessing, and an R2 of about 0 means the remaining inputs explain none of the variation in price. This agrees with the EDA, where no feature showed a relationship with price. The near-perfect scores in Section 5 therefore show that the models learned the labeling rules, not real-market behavior.

7. Streamlit application

The app (`app/app.py`) has three tabs.

- **Advisor:** a form for city, property type, availability, BHK, size, price, year built, floors, facing, furnishing, schools, hospitals, public transport, parking, security, amenities and listing type. After the user clicks Analyze, it shows the verdict (good investment or not), the model's confidence, the estimated price after 5 years with growth rates, the price per sqft compared with the city median, the infrastructure score and a year-by-year growth chart. An expandable box explains the rule behind the verdict.
- **Market insights:** filters for state, BHK and price range, summary figures, price per sqft by state, a state-by-BHK heatmap, the top 10 cities by price and a trend by year built.
- **About the model:** the model types and feature-importance charts for both models.

A separate module, `app/invest_predictor.py`, converts the form answers into the exact 71 columns the models expect, repeating the preprocessing steps above. It was tested against 2,000 real properties (`src/step5_test_predictor.py`): all 2,000 Good Investment labels matched (100.00%), and the future-price error averaged 0.334 lakhs with a worst case of 3.014 lakhs. Three manual tests in the app also behaved as the rule predicts: a property priced below its city median with strong infrastructure was rated a good investment, while one priced above the median, and one with weak infrastructure, were not.

8. Project structure and how to run

```
app/
  app.py                    Streamlit app (3 tabs)
  invest_predictor.py       turns form answers into the 71 model inputs
src/
  step1.py                  preprocessing, features, targets, split, scaling
  step3_classification.py   trains and compares classifiers
  step3_regression.py       trains and compares regressors
  step4_mlflow.py           trains all six models with MLflow tracking
  step5_make_artifacts.py   saves city medians and city-state lookups
  step5_test_predictor.py   checks the app's predictor against real rows
  step6_option_b.py         Option B experiment
  make_requirements.py      writes requirements.txt from installed versions
  compress_models.py        shrinks the saved model files
notebooks/
  eda.ipynb                 the 20 EDA questions
data/                       raw dataset and generated CSV files
models/                     trained models, scaler and lookup files
docs/                       this document and its figures
requirements.txt
README.md
```

Run every command from the project root (the folder that contains `app/`, `data/` and `models/`):

```
python src/step1.py
python src/step3_classification.py
python src/step3_regression.py
python src/step4_mlflow.py
python src/step5_make_artifacts.py
python src/step5_test_predictor.py
python -m streamlit run app/app.py
python -m mlflow ui --backend-store-uri sqlite:///mlflow.db
```

9. Limitations

- **Synthetic data.** Prices, sizes and locations are independent of each other, and the 500 locality names repeat in every city. The data cannot support claims about real housing markets.
- **Defined targets.** The growth formula and the Good Investment rule are assumptions chosen for this project. They are not observed outcomes.
- **Near-perfect scores.** They measure how well the models reproduce the rules, not how well they predict real investments (see Section 6).
- **Floor repair.** Setting Total_Floors to at least Floor_No for 46.5% of rows was a judgment call, and it distorted Floor_Position.
- **Not financial advice.** The application is a learning project and says so on every page.

10. Conclusion and future work

The project delivers a complete pipeline: cleaning and feature engineering, a 20-question exploratory analysis, six tracked models, an application that turns a few inputs into predictions, and an experiment that shows honestly what the models have and have not learned. The most useful lesson is the contrast between Sections 5 and 6: strong scores can come from the way a target is defined, so a model should always be tested without the inputs that define its target.

To make the advisor useful in practice, future work should:

1. Replace the synthetic data with real listings that include sale prices over time.
2. Replace the assumed growth formula with observed appreciation for each location.
3. Define a good investment from real outcomes, such as realized returns or rental yield.
4. Deploy the application (for example on Streamlit Cloud).